

Ch14: Insertion Sort

Slide 1: Understanding Insertion Sort

Insertion Sort is a sorting algorithm that builds the sorted array **one element at a time**.

Instead of trying to sort the entire array at once, it starts with a small sorted portion and gradually grows it.

A useful analogy is **arranging playing cards in your hand**.

Imagine you already have:

3 5

and you pick up a new card:

4

You look at the cards already arranged and place 4 between 3 and 5:

3 4 5

Insertion Sort works in a similar way.

Consider:

5 3 8 2

Initially, we can treat the first element as already sorted:

[5] [3 8 2]

Then we take 3 and insert it into the correct position:

[3 5] [8 2]

Next, 8 is considered:

[3 5 8] [2]

Finally, 2 is inserted into the correct position:

[2 3 5 8]

The Main Idea

Take one element



Compare with sorted portion



Move larger elements right



Insert the element



Sorted portion becomes larger

Unlike Bubble Sort, which repeatedly compares neighboring elements, Insertion Sort focuses on **taking one element and finding where it belongs**.

Key idea:

Insertion Sort grows a sorted portion by inserting one element into its correct position.

Slide 2: Inserting an Element

Let's trace the beginning of the process.

Start with:

5 3 8 2

Treat 5 as the first sorted element:

[5] [3 8 2]

Now select 3.

key = 3

The **key** is the element we are currently trying to place.

Compare 3 with the element before it:

5 3

We ask:

Is $5 > 3$?

Yes.

Since **5** is larger than **3**, it needs to move one position to the right.

Before:

5 3

Move 5 right:

5 5

Now there is an empty position where **3** can be inserted:

3 5

So the array becomes:

3 5 8 2

The sorted portion has now grown:

[3 5] [8 2]

Why Do We Move Instead of Immediately Swapping?

Insertion Sort normally **shifts larger elements to the right** rather than repeatedly swapping neighboring elements.

For example:

5 3

becomes:

5 5

and then:

3 5

This makes room for the selected value.

Another Insertion

Now take **8**:

[3 5] [8 2]

Compare 8 with 5:

$5 > 8$

False.

So 8 is already in the correct position:

[3 5 8] [2]

Key idea:

The selected element is called the **key**, and larger elements are shifted right until the correct position for the key is found.

Slide 3: The Sorted Portion

One of the most important ideas in Insertion Sort is the division between the **sorted portion** and the **unsorted portion**.

We can visualize the array like this:

[sorted portion] [unsorted portion]

Starting array:

5 3 8 2

At the beginning:

[5] [3 8 2]

After inserting 3:

[3 5] [8 2]

After considering 8:

[3 5 8] [2]

After inserting 2:

[2 3 5 8]

Step-by-Step View

Step	Sorted portion	Unsorted portion
Start	5	3 8 2
Insert 3	3 5	8 2
Insert 8	3 5 8	2
Insert 2	2 3 5 8	Empty

Notice something important:

The sorted portion always grows by **one element**.

[5] [3 8 2]

↓

[3 5] [8 2]

↓

[3 5 8] [2]

↓

[2 3 5 8]

This is the central mental model for Insertion Sort.

Visualizing the Movement

For:

3 5 8 2

take 2:

[3 5 8] [2]

Compare 2 with 8:

$8 > 2$

Shift 8 right:

3 5 8 8

Compare 2 with 5:

5 > 2

Shift 5 right:

3 5 5 8

Compare 2 with 3:

3 > 2

Shift 3 right:

3 3 5 8

Now insert 2:

2 3 5 8

The key did not magically teleport into place. The larger elements moved out of its way, because apparently even numbers need a little traffic management.

Key idea:

Insertion Sort maintains a sorted prefix and repeatedly inserts the next element into that prefix.

Slide 4: Insertion Sort in C

The basic C structure is:

```
for (int i = 1; i < n; i++)
{
    int key = arr[i];
    int j = i - 1;

    while (j >= 0 && arr[j] > key)
    {
        arr[j + 1] = arr[j];
        j--;
    }
}
```

```
}  
  
    arr[j + 1] = key;  
}
```

Let's understand each part.

Step 1: Start from the Second Element

```
for (int i = 1; i < n; i++)
```

Why does **i** start at **1**?

Because the first element is treated as the initial sorted portion.

Index:

0 1 2 3

Array:

5 3 8 2

↑

sorted

So we start with the element at index **1**.

Step 2: Store the Current Element

```
int key = arr[i];
```

The selected element is stored in **key**.

For example:

```
arr = 5 3 8 2
```

↑

key

```
key = 3
```

We need to protect this value while other elements are shifted.

Step 3: Start Comparing from the Previous Element

```
int j = i - 1;
```

If:

`i = 1`

then:

`j = 0`

So `j` points to the element immediately before the key.

Step 4: Shift Larger Elements

```
while (j >= 0 && arr[j] > key)
{
    arr[j + 1] = arr[j];
    j--;
}
```

There are two conditions.

`j >= 0`

This makes sure we do not move beyond the beginning of the array.

`arr[j] > key`

This checks whether the current element is larger than the key.

If it is larger, move it one position right:

```
arr[j + 1] = arr[j];
```

Then move `j` left:

```
j--;
```

Step 5: Insert the Key

After all larger elements have moved:

```
arr[j + 1] = key;
```

This places the key into the empty position.

For example:

Before:

3 5 8 2

key = 2

After shifting:

3 3 5 8

Then:

`arr[j + 1] = key;`

Result:

2 3 5 8

Code Flow

Select key



Look at element before key



Is it larger than key?



Yes



Shift it right



Move left



Compare again



Find correct position



Insert key

Key idea:

key stores the element being inserted, **j** searches backward, larger elements shift right, and finally the key is placed into the empty position.

Slide 5: Complete Insertion Sort Process

Let's put everything together using:

5 3 8 2

Step 1: Start

[5] [3 8 2]

5 is considered sorted.

Step 2: Insert 3

5 3

Compare:

$5 > 3$

Shift 5 right:

5 5

Insert 3:

3 5

Array:

[3 5] [8 2]

Step 3: Insert 8

Compare 8 with 5:

$5 > 8$

False.

So no shifting is required:

[3 5 8] [2]

Step 4: Insert 2

Start:

[3 5 8] [2]

Compare with 8:

$8 > 2$

Shift:

3 5 8 8

Compare with 5:

$5 > 2$

Shift:

3 5 5 8

Compare with 3:

$3 > 2$

Shift:

3 3 5 8

Now insert 2:

2 3 5 8

Complete Visual

Start:

[5] [3 8 2]

Insert 3:

[3 5] [8 2]

Insert 8:

[3 5 8] [2]

Insert 2:

[2 3 5 8]

Insertion Sort Mental Model

Select one element



Call it key



Look left



Shift larger elements right



Find correct position



Insert key



Sorted portion grows

Core Takeaway

Insertion Sort builds the sorted array one element at a time.

It selects a **key**, compares it with previous elements, shifts larger values to the right, and inserts the key into its correct position.

The three ideas students should remember are:

Key → element being inserted

Shift → move larger elements right

Insert → place key in the correct position